React debug és adatbázis-kezelés

Tanulási jegyzet: hibakeresés lépésről lépésre, hibatípusok javítással, majd feladatok Context-tel (megjelenítés, hozzáadás, törlés)

Tartalom

- **I. rész – Debug:** hol keresd a hibát, hibatípusok, mit írj és hova
- **II. rész – 6 feladat:** adatok megjelenítése adatbázisból Context-tel, hozzáadás, törlés, teljes megoldásokkal, plusz egy önálló gyakorlófeladat

I. rész – Debug

1. Hol látod a hibákat? (5 hely)

Minden hibának megvan a „saját helye”. Ha tudod, melyik típusú a hiba, tudod, hova nézz. A böngésző fejlesztői eszközeit (DevTools) az **F12** gombbal vagy jobb klikk → **Vizsgálat / Inspect** menüvel nyitod meg.

Hely	Hogyan nyitod meg	Mit látsz itt
1. Console fül (böngésző)	F12 → Console	Piros hibák és sárga warningok a futás közben, a console.log kiírásai. A hiba jobb oldalán lévő **fájlnév:sorszám** linkre kattintva a pontos sorhoz ugrasz.
2. Network fül (böngésző)	F12 → Network → szűrő: **Fetch/XHR**	Minden szerverkérés: URL, metódus, **státuszkód** (200, 404, 500). Kérésre kattintva: **Headers** (cím, metódus), **Payload** (amit küldtél), **Response / Preview** (amit a szerver visszaadott).
3. Terminál (frontend)	Ahol az **npm run dev** fut	Fordítási hibák: rossz import, szintaktikai hiba. Vite ilyenkor a böngészőben is mutat egy piros hibaablakot (overlay).
4. Terminál (backend)	Ahol a szerver fut (node, php artisan serve)	Szerveroldali hibák: adatbázis-kapcsolat, SQL hiba, 500-as hiba oka. Ha a Network 500-at mutat, mindig itt keresd az okot.
5. React DevTools	Böngészőbővítmény, F12 → **Components** fül	A komponensfa, és kijelölve minden komponens **props**, **state** (hooks) és a Context aktuális értéke.

Plusz a **VS Code**: a piros hullámos aláhúzás és a **Problems** panel (Ctrl+Shift+M) már mentés előtt jelzi a szintaktikai hibák és elírások nagy részét.

2. A debug lépéssor – mindig ebben a sorrendben

1. **Van piros hibaablak az oldalon, vagy hiba a frontend terminálban?** → Fordítási hiba. Olvasd el a fájlnevet és a sorszámot, ott a hiba.
2. **Nézd meg a Console-t.** Van piros sor? Olvasd el az üzenetet (lásd 3. fejezet), kattints a fájl:sor linkre.
3. **Ha az adat nem jelenik meg:** Network fül → Fetch/XHR. Elment a kérés? Mi a státuszkód? Mit ad vissza a Response?
4. **Ha a Network 500-at mutat:** nézd meg a backend terminált, ott az igazi hibaüzenet.
5. **Ha nincs hibaüzenet, csak rosszul működik:** tegyél console.log-okat (4. fejezet), és React DevTools-ban nézd meg a state-et.

6. **Javítás után:** mentsd el, frissítsd az oldalt, és nézd meg újra a Console-t.

3. Hibatípusok

Négy nagy csoport van. Mindegyiknél: **milyen üzenet**, **hol látod**, **miért van**, és **mit írj, hova**. A piros keretes kód a hibás, a zöld a javított.

A) Szintaktikai / fordítási hibák

Az alkalmazás el sem indul, vagy fehér az oldal piros hibaablakkal. Ezeket a legkönnyebb javítani, mert pontos sort mutatnak.

„Failed to resolve import ./components/TermekLista”

Hol látod: frontend terminál + Vite hibaablak a böngészőben

Miért: rossz az elérési út, el van írva a fájlnév, vagy hiányzik a kiterjesztés/mappa. Kis- és nagybetű számít!

Hibás:

```
import TermekLista from "./component/termekLista";
```

Javítás – az App.jsx tetején, az importnál. Nézd meg, pontosan mi a mappa és a fájl neve.

```
import TermekLista from "./components/TermekLista";
```

„Adjacent JSX elements must be wrapped in an enclosing tag”

Hol látod: Vite hibaablak, VS Code piros aláhúzás

Miért: a return csak **egy** elemet adhat vissza.

Hibás:

```
return (
  <h1>Termékek</h1>
  <TermekLista />
);
```

Javítás – a komponens return részében, tedd egy <> (Fragment) vagy egy közé.

```
return (
  <>
    <h1>Termékek</h1>
    <TermekLista />
  </>
);
```

„Unexpected token” / „Expected ...”

Hol látod: Vite hibaablak, megadja a sort és oszlopot

Miért: hiányzó vagy felesleges zárójel, kapcsos zárójel, vessző.

Javítás – a jelzett sorban vagy közvetlenül előtte.

```
// Nézd meg a jelzett sort ÉS az előtte lévőt is!
// Gyakori: hiányzó } egy függvény végén, vagy ) a return után.
// VS Code-ban kattints egy zárójelre: kiemeli a párját.
```

B) Futásidejű hibák (Console)

Elindul az oldal, de egy ponton összeomlik vagy fehér lesz. A Console-ban piros üzenet jelenik meg.

„Cannot read properties of undefined (reading 'map')”

Hol látod: Console, a link a komponens sorára mutat

Miért: a .map()-et olyan változón hívod, ami még undefined, mert az adat még nem jött meg, vagy a state kezdőértéke nincs megadva.

Hibás:

```
const [termekek, setTermekek] = useState();   // undefined!
...
{termekek.map(t => ...)}
```

Javítás – a Context fájlban a useState-nél, vagy a komponensben a .map-nél.

```
// 1. A Contextben: a kezdőérték legyen üres tömb
const [termekek, setTermekek] = useState([]);

// 2. Vagy a komponensben védd le ?. -tal:
{termekek?.map(t => <TermekSor key={t.id} termek={t} />)}
```

„useTermekek is not a function” / „... is not defined”

Hol látod: Console

Miért: nem importáltad, rossz néven importáltad, vagy named export helyett default importot használsz.

Hibás:

```
import useTermekek from "../context/TermekContext";
```

Javítás – a komponens tetején. „export function” → kapcsos zárójellel; „export default” → anélkül.

```
import { useTermekek } from "../context/TermekContext";
```

„useTermekek-et csak TermekProvider-en belül lehet használni!”

Hol látod: Console (ezt a hibát mi magunk dobjuk a hookban)

Miért: a komponens nincs a Provider-en belül, ezért nem éri el a contextet.

Hibás:

```
<TermekProvider>
  <UjTermek />
</TermekProvider>
<TermekLista />   // kívül van!
```

Javítás – az App.jsx-ben: minden olyan komponens legyen a Provider között, ami használja.

```
<TermekProvider>
  <UjTermek />
  <TermekLista />
</TermekProvider>
```

„Too many re-renders” / végtelen kéréssorozat

Hol látod: Console, illetve a Network fülön folyamatosan jönnek az új kérések

Miért: a renderelés közben állítasz state-et, vagy a useEffect-ből hiányzik a függőségi tömb.

Hibás:

```
useEffect(() => {
  termekekBetoltese();
});                 // nincs [] → minden render után lefut
```

Javítás – a Context fájlban, a useEffect végén.

```
useEffect(() => {
  termekekBetoltese();
}, []);             // [] → csak egyszer fut le
```

A törlés gomb magától lefut betöltéskor

Hol látod: Network fül: rögtön DELETE kérések mennek; a lista kiürül

Miért: zárójellel a függvény azonnal meghívódik, nem a kattintáskor.

Hibás:

```
<button onClick={termekTorlese(termek.id)}>Törlés</button>
```

Javítás – a gomb onClick-jénél. Ha paramétert adsz át, kell az () => nyílfüggvény.

```
<button onClick={() => termekTorlese(termek.id)}>Törlés</button>
```

„Objects are not valid as a React child”

Hol látod: Console

Miért: egy egész objektumot próbálsz kiírni a JSX-ben.

Hibás:

```
<td>{termek}</td>
```

Javítás – a JSX-ben: mindig az objektum egy mezőjét írd ki.

```
<td>{termek.nev}</td>
```

C) Hálózati / API hibák (Network fül)

Ezeknél az oldal működik, csak az adat nem jön meg, nem mentődik, vagy nem törlődik.
Mindig a Network fülön kezdd: kattints a kérésre, és nézd meg a státuszkódot és a Response-t.

Amit látsz	Mit jelent	Mit csinálj
Console: **Failed to fetch**, Network: **(failed)**	A szerver nem fut, vagy rossz a cím/port	Indítsd el a backendet. Nyisd meg az URL-t a böngészőben: ha ott sem jön adat, a backend a hibás.
Console: **CORS policy** hiba	A backend nem engedi, hogy más portról kérj adatot	Backendben kell engedélyezni (Express: cors csomag, Laravel: config/cors.php).
404 Not Found	Rossz URL / végpont	Hasonlítsd össze a fetch címét a backend route-jával (pl. /api/termekek vs /api/termek).

Amit látsz	Mit jelent	Mit csinálj
400 / 422	Rossz vagy hiányzó adat a kérésben	Network → Payload: nézd meg, mit küldtél. Van Content-Type header? JSON.stringify-oltad?
500 Internal Server Error	A szerveren történt hiba	A backend terminálban van az igazi üzenet (SQL hiba, rossz mezőnév, adatbázis nem fut).
Unexpected token '<' ... is not valid JSON	JSON helyett HTML jött vissza (hibaoldal vagy rossz URL)	Network → Response: ha HTML-t látsz, rossz az URL vagy a szerver hibát adott.

A fetch nem jelez hibát 404-nél vagy 500-nál

Hol látod: a Console-ban nincs hiba, de nincs adat sem

Miért: a fetch csak akkor dob hibát, ha egyáltalán nem éri el a szervert. A 404 és 500 „sikeres” válasznak számít.

Hibás:

```
const valasz = await fetch("http://localhost:3000/api/termekek");
const adat = await valasz.json();
```

Javítás – a Context minden fetch-e után, a .json() előtt.

```
const valasz = await fetch("http://localhost:3000/api/termekek");
if (!valasz.ok) {
  throw new Error(`Szerver hiba: ${valasz.status}`);
}
const adat = await valasz.json();
```

A POST elmegy, de a szerver üres adatot kap

Hol látod: Network → Payload üres vagy [object Object]; a szerver 400-at vagy 422-t ad

Miért: hiányzik a header vagy a JSON.stringify.

Hibás:

```
fetch(url, { method: "POST", body: ujTermek });
```

Javítás – a Context termekHozzaadasa függvényében.

```
fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(ujTermek),
});
```

D) Logikai hibák – nincs hibaüzenet, mégsem jó

A legnehezebbek, mert semmi nem piros. Itt a console.log és a React DevTools segít.

Hozzáadás/törlés után a lista nem frissül

Hol látod: Network: a kérés 200/201, a szerveren megvan a változás, de a képernyőn nem

Miért: a state-et közvetlenül módosítod (push, splice), vagy elfelejtetted frissíteni. A React csak **új** tömbre renderel újra.

Hibás:

```
termekek.push(mentett);          // a React nem veszi észre
setTermekek(termekek);
```

Javítás – a Context függvényeiben, a sikeres fetch után.

```
setTermekek(elozo => [...elozo, mentett]);          // hozzáadás
setTermekek(elozo => elozo.filter(t => t.id !== id)); // törlés
```

A console.log a régi értéket írja ki

Hol látod: Console

Miért: a setState nem azonnal változtatja meg a változót, csak a következő renderben.

Hibás:

```
setTermekek(adat);
console.log(termekek);   // még a RÉGI érték!
```

Javítás – a Context fájlban, a useEffect-et a többi alá.

```
// Vagy az új adatot logold:
console.log(adat);

// Vagy figyeld a változást useEffect-tel:
useEffect(() => {
  console.log("termekek változott:", termekek);
}, [termekek]);
```

Az ár szövegként mentődik („1000” helyett 1000)

Hol látod: Network → Payload: az ar értéke idézőjelben van

Miért: az input értéke mindig string.

Hibás:

```
termekHozzaadasa({ nev, ar });
```

Javítás – az UjTermek komponens mentes függvényében.

```
termekHozzaadasa({ nev: nev.trim(), ar: Number(ar) });
```

Warningok (sárga sorok a Console-ban)

Warning	Javítás és hely
Each child in a list should have a unique "key" prop	A .map()-ben a visszaadott elemre: key={termek.id} (TermekLista.jsx)
A component is changing an uncontrolled input to be controlled	A useState kezdőértéke legyen "", ne undefined (UjTermek.jsx)
React Hook useEffect has a missing dependency	Tedd be a függőséget a [ ] tömbbe, vagy ha szándékosan egyszer fut, hagyd így (ESLint figyelmeztetés)

4. Debug kódrészletek – mit írj és pontosan hova

Ezeket ideiglenesen teszed be a kódba, hogy lásd, mi történik. Ha megtaláltad a hibát, töröld őket.

4.1 A szerver válaszának ellenőrzése

Hova: a Context fájlban a termekekBetoltese függvénybe, a fetch után. Így kiderül, hogy a szerver válaszol-e, és milyen adatot küld.

```
async function termekekBetoltese() {
  try {
    console.log("1. Kérés indul...");
    const valasz = await fetch("http://localhost:3000/api/termekek");
    console.log("2. Státusz:", valasz.status, valasz.ok);

    const adat = await valasz.json();
    console.log("3. Kapott adat:", adat);
    console.table(adat);              // táblázatban mutatja a tömböt

    setTermekek(adat);
  } catch (err) {
    console.error("4. HIBA:", err);   // piros sorként jelenik meg
  }
}
```

Hogyan olvasd: ha csak az 1. jelenik meg, utána a 4., a szerver nem elérhető. Ha a 2. státusza 404 vagy 500, a cím vagy a backend hibás. Ha a 3. adat nem tömb (pl. `{ data: [...] }`), akkor `setTermekek(adat.data)` kell.

4.2 Ha nem JSON jön vissza

Hova: ugyanoda, a .json() helyére, ideiglenesen. Megmutatja nyersen, mit küldött a szerver.

```
const valasz = await fetch("http://localhost:3000/api/termekek");
const szoveg = await valasz.text();
console.log("Nyers válasz:", szoveg);   // ha <!DOCTYPE html> → rossz URL vagy szerverhiba
```

4.3 Mit kap a komponens?

Hova: a komponens függvényén belül, a return elé.

```
export default function TermekLista() {
  const { termekek, betoltes, hiba } = useTermekek();
  console.log("TermekLista render:", { termekek, betoltes, hiba });

  return ( ... );
}
```

4.4 Lefut-e a függvény? (kattintás, űrlap)

Hova: az eseménykezelő függvény első sorába. Ha nem jelenik meg semmi a Console-ban, a gomb nincs jól bekötve.

```
async function mentes(e) {
  e.preventDefault();
  console.log("Mentés gomb megnyomva:", { nev, ar });
  ...
}

function torlesKattintas() {
  console.log("Törlés:", termek.id);
  ...
}
```

4.5 Megállítás egy ponton: debugger

Hova: bárhová, ahol meg akarod állítani a futást. Nyitott DevTools mellett a böngésző ott megáll, és a **Sources** fülön egérrel a változók fölé állva látod az értéküket. A **F10** egy sort lép tovább, az **F8** folytatja a futást.

```
const adat = await valasz.json();
debugger;            // itt megáll – nézd meg az "adat" értékét
setTermekek(adat);
```

4.6 Hiba kiírása a felhasználónak

Ez nem ideiglenes, ez maradjon a kódban: a hibát a state-be teszed, és a komponens kiírja.
Hova: a Contextben a catch ágba, a komponensben a return elé.

```
// Contextben:
const [hiba, setHiba] = useState(null);
...
} catch (err) {
  console.error("Betöltési hiba:", err);
  setHiba(err.message);
}

// Komponensben (TermekLista.jsx):
if (hiba) return <p style={{ color: "red" }}>Hiba: {hiba}</p>;
```

Debug szabályok röviden:

- Mindig a **legelső** piros hibát javítsd, a többi gyakran csak következmény.
- Olvasd el a hibaüzenetet szó szerint, és nézd meg a fájl:sor linket.
- Adat nem jön → **Network**. 500-as hiba → **backend terminál**. Nincs hiba, de rossz → **console.log + React DevTools**.
- A console.log-okat számozd (1., 2., 3.), így látod, meddig jutott a kód.
- Javítás után töröld a debug console.log-okat és a debugger sorokat.

II. rész – 6 feladat: adatok adatbázisból Context-tel

Hat feladat, mindegyik egy fájl elkészítése. Együtt egy kész alkalmazást adnak, ami megjeleníti az adatbázis adatait, és lehet hozzáadni és törölni. Mindegyiknél előbb a teendők, utána a teljes megoldás. Próbáld meg előbb önállóan!

Kiinduló helyzet

A backend már kész, és ezeket a végpontokat adja (pl. Node.js/Express vagy Laravel):

Metódus	Végpont	Mit csinál
GET	`/api/termekek`	Visszaadja az összes terméket (tömb)
POST	`/api/termekek`	Elment egy újat, visszaadja id-val együtt
DELETE	`/api/termekek/:id`	Törli az adott id-jú terméket

Egy termék JSON-ban: `{ "id": 1, "nev": "Laptop", "ar": 250000 }`

Fájlszerkezet

```
src/
├── main.jsx                 → 6. feladat
├── App.jsx                  → 5. feladat
├── context/
│   └── TermekContext.jsx    → 1. feladat
└── components/
    ├── TermekLista.jsx      → 2. feladat
    ├── TermekSor.jsx        → 3. feladat
    └── UjTermek.jsx         → 4. feladat
```

A szerver címe: kétféle írásmód

A megoldásokban a cím **közvetlenül a fetch-be van írva**. Mindenhol alatta, **kommentben** ott a változós változat, ami élesben a helyes, mert ha a cím változik, csak egy helyen kell átírni.

```
// Közvetlenül (ez fut):
const valasz = await fetch("http://localhost:3000/api/termekek");

// Változóval (kommentben):
// const API_URL = "http://localhost:3000";
// const TERMEKEK_URL = `${API_URL}/api/termekek`;
// const valasz = await fetch(TERMEKEK_URL);

// Vite + .env fájl (projekt gyökere: VITE_API_URL=http://localhost:3000):
// const API_URL = import.meta.env.VITE_API_URL;
```

1. feladat – A Context fájl

Fájl: `src/context/TermekContext.jsx`

Teendők:

- Hozz létre egy contextet a **createContext**-tel.
- A **TermekProvider**-ben három state: termekek (kezdőérték: üres tömb), betoltes, hiba.
- **termekekBetoltese**: GET kérés, ellenőrizd a valasz.ok-ot, az adatot tedd a state-be. A **useEffect** hívja meg egyszer.
- **termekHozzaadasa(ujTermek)**: POST kérés headerrel és JSON.stringify-jal, az új elemet tedd a lista végére.
- **termekTorlese(id)**: DELETE kérés az id-val a címben, az elemet szűrd ki a .filter()-rel.
- Mindent adj át a Provider **value**-jában, és készíts egy **useTermekek** hookot.
- A szerver címét írd közvetlenül a fetch-be, a változós változatot pedig kommentbe.

Megoldás

```
import { createContext, useContext, useEffect, useState } from "react";

// ---------------------------------------------------------------
// VÁLTOZÓS VERZIÓ (kommentben) – így szokás élesben:
//
// const API_URL = "http://localhost:3000";
// // vagy Vite-ban .env fájlból: VITE_API_URL=http://localhost:3000
// // const API_URL = import.meta.env.VITE_API_URL;
// const TERMEKEK_URL = `${API_URL}/api/termekek`;
// ---------------------------------------------------------------

// 1. Létrehozzuk a contextet
const TermekContext = createContext();

// 2. Provider: ez tárolja az adatokat, és átadja minden gyereknek
export function TermekProvider({ children }) {
  const [termekek, setTermekek] = useState([]);
  const [betoltes, setBetoltes] = useState(true);
  const [hiba, setHiba] = useState(null);

  // --- Adatok lekérése az adatbázisból (GET) ---
  async function termekekBetoltese() {
    try {
      setBetoltes(true);
      const valasz = await fetch("http://localhost:3000/api/termekek");
      // Változós verzió:
      // const valasz = await fetch(TERMEKEK_URL);

      if (!valasz.ok) {
        throw new Error(`Szerver hiba: ${valasz.status}`);
      }
      const adat = await valasz.json();
      setTermekek(adat);
      setHiba(null);
    } catch (err) {
      console.error("Betöltési hiba:", err);
      setHiba(err.message);
    } finally {
      setBetoltes(false);
    }
  }

  // Első megjelenéskor egyszer lefut
  useEffect(() => {
    termekekBetoltese();
  }, []);

  // --- Új elem hozzáadása (POST) ---
  async function termekHozzaadasa(ujTermek) {
    try {
      const valasz = await fetch("http://localhost:3000/api/termekek", {
        // Változós verzió:
        // const valasz = await fetch(TERMEKEK_URL, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(ujTermek),
      });

      if (!valasz.ok) {
        throw new Error(`Mentési hiba: ${valasz.status}`);
```

```
      }
      const mentett = await valasz.json(); // a szerver visszaadja az id-val együtt
      setTermekek((elozo) => [...elozo, mentett]);
    } catch (err) {
      console.error("Hozzáadási hiba:", err);
      setHiba(err.message);
    }
  }

  // --- Elem törlése (DELETE) ---
  async function termekTorlese(id) {
    try {
      const valasz = await fetch(`http://localhost:3000/api/termekek/${id}`, {
        // Változós verzió:
        // const valasz = await fetch(`${TERMEKEK_URL}/${id}`, {
        method: "DELETE",
      });

      if (!valasz.ok) {
        throw new Error(`Törlési hiba: ${valasz.status}`);
      }
      setTermekek((elozo) => elozo.filter((t) => t.id !== id));
    } catch (err) {
      console.error("Törlési hiba:", err);
      setHiba(err.message);
    }
  }

  return (
    <TermekContext.Provider
      value={{ termekek, betoltes, hiba, termekHozzaadasa, termekTorlese }}
    >
      {children}
    </TermekContext.Provider>
  );
}
// 3. Saját hook, hogy a komponensekben egyszerű legyen használni
export function useTermekek() {
  const context = useContext(TermekContext);
  if (!context) {
    throw new Error("A useTermekek-et csak TermekProvider-en belül lehet használni!");
  }
  return context;
}
```

Ellenőrzés és debug:

- Network fül: GET kérés 200-as státusszal, a Response-ban a termékek tömbje.
- React DevTools → TermekProvider: a termekek state-ben ott vannak az adatok.
- Ha „Betöltés...” marad örökre: van-e finally ág, és lefut-e a useEffect.

2. feladat – Lista megjelenítése

Fájl: `src/components/TermekLista.jsx`

Teendők:

- A **useTermekek()** hookkal kérd le: termekek, betoltes, hiba.
- Kezeld a négy állapotot: betöltés, hiba (piros szöveg), üres lista, kész lista.
- A termékeket táblázatban jelenítsd meg: a **.map()** minden termékből egy TermekSor-t csináljon, **key={termek.id}**-val.

Megoldás

```
import { useTermekek } from "../context/TermekContext";
import TermekSor from "./TermekSor";

// Megjeleníti az adatbázisból jött adatokat
export default function TermekLista() {
  const { termekek, betoltes, hiba } = useTermekek();

  if (betoltes) return <p>Betöltés...</p>;
  if (hiba) return <p style={{ color: "red" }}>Hiba: {hiba}</p>;
  if (termekek.length === 0) return <p>Még nincs termék. Adj hozzá egyet!</p>;

  return (
    <table>
      <thead>
        <tr>
          <th>Név</th>
          <th>Ár</th>
          <th></th>
        </tr>
      </thead>
      <tbody>
        {termekek.map((termek) => (
          // A key kötelező listáknál, különben warning jön a konzolban
          <TermekSor key={termek.id} termek={termek} />
        ))}
      </tbody>
    </table>
  );
}
```

Ellenőrzés és debug:

- Ha „Cannot read properties of undefined (reading 'map')”: a Contextben a useState kezdőértéke legyen [].
- Ha „unique key” warning: hiányzik a key a .map()-ben.

3. feladat – Egy sor törléssel

Fájl: `src/components/TermekSor.jsx`

Teendők:

- Propsként kapja meg a **termek** objektumot, a contextből pedig a **termekTorlese** függvényt.
- Jelenítse meg a nevet és az árat egy táblázatsorban.
- Legyen egy Törlés gomb, ami előtte megerősítést kér (**window.confirm**).

Megoldás

```
import { useTermekek } from "../context/TermekContext";

// Egy sor a táblázatban, törlés gombbal
export default function TermekSor({ termek }) {
  const { termekTorlese } = useTermekek();

  function torlesKattintas() {
    if (window.confirm(`Biztosan törlöd: ${termek.nev}?`)) {
      termekTorlese(termek.id);
    }
  }
  return (
    <tr>
      <td>{termek.nev}</td>
      <td>{termek.ar} Ft</td>
      <td>
        {/* Figyelem: onClick={torlesKattintas}, NEM onClick={torlesKattintas()} */}
        <button onClick={torlesKattintas}>Törlés</button>
      </td>
    </tr>
  );
}
```

Ellenőrzés és debug:

- Network: DELETE kérés a /api/termekek/3 címre. Ha /undefined van a végén, rossz az id átadása.
- Ha betöltéskor magától töröl: onClick-ben zárójellel hívod a függvényt.
- Ha a szerveren törlődik, de a képernyőn nem: hiányzik a setTermekek a filterrel.

4. feladat – Új elem hozzáadása

Fájl: `src/components/UjTermek.jsx`

Teendők:

- Két state az inputokhoz: **nev** és **ar**, kezdőérték: üres string.
- Az űrlap küldésekor: **e.preventDefault()**, ellenőrizd, hogy nincs-e üres mező.
- Hívd meg a **termekHozzaadasa**-t, az árat **Number()**-rel alakítsd számmá, utána ürítsd ki a mezőket.

Megoldás

```
import { useState } from "react";
import { useTermekek } from "../context/TermekContext";

// Új termék hozzáadása űrlappal
export default function UjTermek() {
  const { termekHozzaadasa } = useTermekek();
  const [nev, setNev] = useState("");
  const [ar, setAr] = useState("");

  async function mentes(e) {
    e.preventDefault(); // ne töltse újra az oldalt
    if (!nev.trim() || !ar) {
      alert("A név és az ár kötelező!");
      return;
    }

    await termekHozzaadasa({ nev: nev.trim(), ar: Number(ar) });
    setNev("");
    setAr("");
  }

  return (
    <form onSubmit={mentes}>
      <input
        placeholder="Termék neve"
        value={nev}
        onChange={(e) => setNev(e.target.value)}
      />
      <input
        type="number"
        placeholder="Ár"
        value={ar}
        onChange={(e) => setAr(e.target.value)}
      />
      <button type="submit">Hozzáadás</button>
    </form>
  );
}
```

Ellenőrzés és debug:

- Network: POST kérés, a Payload-ban ott a név és az ár (az ár szám, nem string).
- A válasz 200 vagy 201, a Response-ban az új elem id-val.
- Ha küldéskor újratölt az oldal: hiányzik az e.preventDefault().

5. feladat – Összerakás

Fájl: `src/App.jsx`

Teendők:

- Importáld a TermekProvider-t és a komponenseket.
- A **TermekProvider** vegye körbe az UjTermek és a TermekLista komponenst.

Megoldás

```
import { TermekProvider } from "./context/TermekContext";
import TermekLista from "./components/TermekLista";
import UjTermek from "./components/UjTermek";

export default function App() {
  return (
    // Minden komponens, ami a Provider-en belül van, eléri az adatokat
    <TermekProvider>
      <h1>Termékek</h1>
      <UjTermek />
      <TermekLista />
    </TermekProvider>
  );
}
```

Ellenőrzés és debug:

- Ha „useTermekek-et csak TermekProvider-en belül...” hiba: egy komponens kívül maradt a Provider-en.
- Ha „Failed to resolve import”: nézd meg a mappa- és fájlnevet (kis- és nagybetű számít).

6. feladat – Belépési pont

Fájl: `src/main.jsx`

Teendők:

- A **createRoot**-tal csatold az App-ot az index.html **root** id-jú elemére.
- Vedd körbe **StrictMode**-dal (fejlesztéskor segít a hibák kiszűrésében).

Megoldás

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Ellenőrzés és debug:

- StrictMode miatt fejlesztéskor a useEffect kétszer fut le, ezért a Network-ben két GET kérést látsz. Ez normális, élesben egyszer fut.
- Ha fehér az oldal hiba nélkül: nézd meg, van-e az index.html-ben `<div id="root">`.

Önálló gyakorlófeladat (megoldás nélkül)

Csináld meg ugyanezt felhasználókra:

- Végpontok: `GET/POST /api/felhasznalok` és `DELETE /api/felhasznalok/:id`. Egy felhasználó: id, nev, email.
- Készíts **FelhasznaloContext.jsx**-et ugyanígy, de most a szerver címét **változóba** tedd (API_URL), és a közvetlen változat legyen kommentben.
- Komponensek: FelhasznaloLista, FelhasznaloSor, UjFelhasznalo.
- Plusz: ha üres az email mező, ne küldje el az űrlapot.
- Szándékosan rontsd el egyenként: rossz URL, hiányzó [] a useEffect-ben, onClick zárójellel. Nézd meg, hol és milyen hibát látsz!

Ellenőrző kérdések:

- Hol nézed meg, ha az adat nem jelenik meg? És ha 500-as hibát kapsz?
- Miért kell az `if (!valasz.ok)`, ha van try/catch?
- Mi a különbség `onClick={torles}` és `onClick={torles()}` között?
- Miért `[...elozo, uj]` és `.filter()`, és miért nem push és splice?
- Miért jobb változóba tenni a szerver címét?